

EventTix

Plataforma de Venta y Gestión de Boletos Inteligentes

Informe Final de Proyecto

Curso:

Desarrollo de Aplicaciones para Dispositivos Móviles

Integrantes:

Roger Infante Asalde

Repositorio de GitHub (Frontend):

<https://github.com/rogerinfas/EventTix-Flutter>

API VPS Backend:

<http://77.42.83.15:9988>

Video de Presentación:

<https://drive.google.com/drive/folders/181GyMDT102DKJVuiSToELLibBGBIhKdH?usp=sharing>

Julio, 2026

Índice

1. Introducción y Público Objetivo	2
1.1. El Problema a Resolver	2
1.2. Público Objetivo	2
1.3. Credenciales de Usuarios de Prueba (Seeds)	2
2. Características Principales de la Aplicación	3
3. Arquitectura del Sistema y Patrón de Diseño	4
3.1. Patrón de Diseño MVVM (Model-View-ViewModel)	4
3.2. Capa de Servicio de Red	4
3.3. Estructura del Backend	4
4. Decisiones de Diseño y Aprendizajes	5
4.1. Decisiones de Diseño Clave	5
4.2. Aprendizajes	5
5. Rúbrica de Evaluación y Autoevaluación	6

1. Introducción y Público Objetivo

1.1. El Problema a Resolver

En la actualidad, la adquisición de boletos para eventos en vivo suele ser un proceso engorroso o propenso a fraudes en la reventa. Los usuarios se enfrentan frecuentemente a los siguientes inconvenientes:

- **Transferir entradas:** Imposibilidad de transferir boletos de forma segura a amigos o familiares sin incurrir en cobros abusivos de plataformas intermediarias.
- **Cancelación de reservas:** Falta de mecanismos rápidos para cancelar boletos cuando surge un imprevisto, impidiendo liberar el asiento a otros interesados en tiempo real.
- **Acceso offline y seguridad:** La necesidad de contar con códigos de barras estáticos o PDFs que son fáciles de duplicar y no brindan una garantía de seguridad integrada al dispositivo.

1.2. Público Objetivo

El público objetivo de **EventTix** se clasifica en dos segmentos fundamentales:

- **Usuarios Finales (Espectadores):** Personas de entre 16 y 60 años habituales al consumo digital, que asisten a conciertos, obras teatrales y eventos deportivos, y exigen transacciones fluidas y seguras.
- **Organizadores de Eventos:** Promotores de entretenimiento que requieren visibilidad en tiempo real sobre el estado de la venta de entradas, cancelaciones y ocupación de asientos en el recinto.

1.3. Credenciales de Usuarios de Prueba (Seeds)

Para facilitar la evaluación de la funcionalidad en tiempo real y flujos P2P, la base de datos se encuentra sembrada con las siguientes cuentas de prueba:

- **Usuario Administrador:**
 - Correo electrónico: `admin@eventtix.com`
 - Contraseña: `admin123`
- **Usuario Secundario:**
 - Correo electrónico: `user@eventtix.com`
 - Contraseña: `user123`

2. Características Principales de la Aplicación

La aplicación móvil de **EventTix** se comunica en tiempo real con una API REST alojada en un servidor VPS, proveyendo las siguientes funcionalidades principales:

1. **Autenticación Segura (JWT):** Flujo de inicio de sesión y registro con tokens web JSON persistidos localmente a través de `shared_preferences`.
2. **Cartelera de Eventos Dinámica:** Listado en tiempo real de los eventos activos con visualización adaptada a dispositivos móviles.
3. **Adquisición Inmediata:** Compra asíncrona de boletos con selección de sección y asiento.
4. **Billetera Inteligente:** Módulo centralizado para ver boletos comprados con ordenación interactiva (activos arriba, inactivos/cancelados al fondo).
5. **Smart Ticket™ (QR Local):** Generación local de códigos QR dinámicos a partir de la firma de seguridad guardada en el backend.
6. **Cancelación en un Toque:** Cancelación inmediata con liberación automática de asientos en el backend.
7. **Transferencia P2P:** Envío directo y seguro de boletos a cualquier otro usuario del sistema buscando únicamente su correo electrónico.

3. Arquitectura del Sistema y Patrón de Diseño

3.1. Patrón de Diseño MVVM (Model-View-ViewModel)

El desarrollo de la aplicación de Flutter se estructuró siguiendo el patrón arquitectónico **MVVM**, garantizando modularidad, desacoplamiento y facilidad de testing:

- **Model (Modelos):** Clases Dart independientes como `Evento` y `Boleto` que procesan la deserialización del formato JSON enviado por la API.
- **View (Vistas):** Componentes declarativos de UI (ej. `PantallaBilletera`) estilizados bajo una paleta oscura y estructurados en widgets pequeños y reutilizables.
- **ViewModel (Modelos de Vista):** Controladores intermedios como `EventosViewModel` y `BilleteraViewModel` que exponen los flujos de datos asíncronos y mantienen el estado de la UI limpio.

3.2. Capa de Servicio de Red

Se implementó un cliente centralizado en la clase `ApiService` que gestiona las peticiones HTTP seguras. Esta clase inyecta automáticamente el token de autorización JWT guardado en el almacenamiento seguro de la app móvil.

3.3. Estructura del Backend

El backend está programado en Python utilizando el microframework Flask y persistiendo los datos en una base de datos relacional SQLite ligera. Los endpoints están desplegados bajo una dirección IP dedicada (`http://77.42.83.15:9988`), permitiendo a la app móvil consultar el catálogo de eventos y realizar las operaciones seguras de billetera.

4. Decisiones de Diseño y Aprendizajes

4.1. Decisiones de Diseño Clave

- **Eliminación de IndexedStack por Ciclo de Vida:** Inicialmente, la navegación de pestañas utilizaba `IndexedStack`, lo que congelaba los sub-widgets en memoria y no llamaba a la API cuando el usuario regresaba de comprar un boleto. Se decidió rediseñar el control de vistas a través de un `switch/case` dinámico con un `ValueKey` para forzar la reconstrucción limpia de la pantalla activa.
- **Procesamiento de Errores P2P en el Cliente:** Al transferir un boleto, en lugar de manejar una validación estática local, la aplicación delega la validación de correos electrónicos al servidor REST. Si el destinatario no está registrado, el frontend captura el error HTTP 404 del backend y lo renderiza de forma elegante en el modal de transferencia.

4.2. Aprendizajes

- **Gestión de Estado Asíncrono:** El uso correcto de `FutureBuilder` en conjunto con recargadores asíncronos (`RefreshIndicator`) permite crear una experiencia mucho más reactiva y de alta fidelidad táctil.
- **Estandarización del Historial de Cambios:** Implementar *Conventional Commits* en el control de versiones facilitó a los desarrolladores comprender qué cambios corregían fallos de regresión y qué cambios agregaban nuevas funcionalidades al backend o frontend.

5. Rúbrica de Evaluación y Autoevaluación

Criterio de Evaluación	Nota	Justificación Técnica y Detalle
1. Funcionalidad general (Navegación, persistencia, UX)	Bueno (5)	Todos los flujos principales (autenticación JWT, listado de cartelera, compra asíncrona, cancelación y transferencia) están operativos, conectados a un VPS activo y con refresco inmediato.
2. Interfaz de usuario (Diseño, maquetación, layouts)	Bueno (5)	Diseño oscuro premium con acentos vibrantes, widget de boleto troquelado, generación dinámica de código QR y adaptabilidad completa.
3. Documentación (Documento LaTeX + video)	Bueno (5)	Presentación modular en LaTeX, documentación detallada paso a paso y enlace al video demostrativo de ejecución.
4. Código y buenas prácticas (Modularidad, commits, git)	Bueno (5)	Patrón arquitectónico MVVM, métodos auxiliares limpios fuera del build, commits semánticos ordenados y README estructurado.
Puntaje Autoevaluado Total	20 / 20 puntos	

Cuadro 1: Tabla de Autoevaluación del Proyecto Final.